問題管理API設計書

メタデータ

- **作成日**: 2026-01-27
- **バージョン**: 1.1.0
- **最終更新**: 2026-03-12
- **担当**: Development Team
- **実装状況**: ✅ 実装済み（テスト103件すべてPASS）

概要

このドキュメントは、実験作業中に発生する問題の報告・管理を行うためのAPI仕様を定義します。

主な機能

- 問題報告（プロジェクト・タスク・トランジション単位）
- コメント投稿による問題のディスカッション
- 問題一覧の取得（複合フィルタリング対応）
- 問題詳細とコメント履歴の取得
- 問題ステータスの変更（未解決/解決済）

対象ユーザー

- 実験作業者（Worker）: 問題報告、コメント投稿、一覧閲覧、ステータス変更
- 実験管理者（Manager）: 上記すべて
- システム管理者（Admin）: 全権限

📋 API一覧

No	HTTPメソッド	エンドポイント	説明	実装状態	権限
1	POST	/api/v1/issues	問題報告	✅ 実装済み	Worker以上
2	POST	/api/v1/issues/comments	コメント投稿	✅ 実装済み	Worker以上
3	GET	/api/v1/issues	問題一覧取得	✅ 実装済み	Worker以上
4	GET	/api/v1/issues/:issueId	問題詳細取得	✅ 実装済み	Worker以上
5	PATCH	/api/v1/issues/status	問題ステータス変更	✅ 実装済み	Worker以上

🔐 共通仕様

認証・認可

認証方式: JWT + Session Hybrid

権限チェック: `checkPermission('050000', permission)`

- 読取（0）: `checkPermission('050000', 0)`
- 書込（1）: `checkPermission('050000', 1)`

ミドルウェアチェーン（共通）:

```
router.METHOD('/path',
  authMiddleware,           // JWT認証
  tenantIsolation,          // テナント分離
  checkPermission('050000', 0 or 1),  // 権限チェック
  [...validationRules],     // バリデーション
  handleValidationErrors,   // バリデーションエラー処理
  issueController.handler   // コントローラー処理
);
```

---

1. 問題報告API

概要

プロジェクト、タスクまたはトランジション単位で問題を報告します。

エンドポイント

```
POST /api/v1/issues
```

認証要件

- JWT認証必須
- ロール: Worker以上（`checkPermission('050000', 1)`）
- テナント分離必須

ミドルウェアチェーン

```
router.post('/',
  authMiddleware,
  tenantIsolation,
  checkPermission('050000', 1),
  [...validationRules],
  handleValidationErrors,
  issueController.create
);
```

リクエスト

リクエストボディ

```
{
  "projectId": "550e8400-e29b-41d4-a716-446655440000",
  "methodId": "method_001",
  "issueType": 1,
  "targetType": "transition",
  "taskNo": 1,
  "transitionId": "transition_001",
  "title": "試薬Aの在庫不足",
```

```
  "description": "実験に必要な試薬Aの在庫が不足しています。追加発注が必要です。"
}
```

パラメータ詳細

パラメータ	型	必須	説明	制約
projectId	string	○	プロジェクトID	文字列
methodId	string	△	メソッドID	オプション
issueType	integer	○	問題の種類	`GET /api/v1/code-master/issue_type/options` で取得した `code_value`（0以上の整数）
targetType	string	○	対象種別	`"project"`, `"task"`, `"transition"` のいずれか
taskNo	integer	△	タスク番号	targetTypeが `"task"` または `"transition"` の場合必須、1以上の整数
transitionId	string	△	トランジションID	targetTypeが `"transition"` の場合必須
title	string	○	問題タイトル	1-200文字
description	string	○	問題詳細	1-2000文字

レスポンス

成功時（201 Created）

```
{
  "success": true,
  "data": {
    "organization_id": "org-uuid",
    "project_id": "550e8400-e29b-41d4-a716-446655440000",
    "issue_id": "issue_001",
    "method_id": "method_001",
    "issue_target": 3,
    "task_no": 1,
    "transition_id": "transition_001",
```

```
    "issue_title": "試薬Aの在庫不足",
    "issue_type": 1,
    "issue_contents": "実験に必要な試薬Aの在庫が不足しています。追加発注が必要です。",
    "status": "unresolved",
    "issue_reporter_id": "880e8400-e29b-41d4-a716-446655440003",
    "issue_report_at": "2026-01-27T10:30:00Z",
    "issue_completer_id": null,
    "issue_complate_at": null,
    "create_at": "2026-01-27T10:30:00Z",
    "update_at": "2026-01-27T10:30:00Z"
  },
  "meta": {
    "timestamp": "2026-01-27T10:30:00Z",
    "requestId": "req_abc123"
  }
}
```


エラー時（400 Bad Request）


```
{
  "success": false,
  "error": {
    "code": "VALIDATION_ERROR",
    "message": "無効な問題タイプです",
    "details": [
      {
        "field": "issueType",
        "message": "Invalid issue type",
        "code": "INVALID_VALUE"
      }
    ]
  },
  "meta": {
    "timestamp": "2026-01-27T10:30:00Z",
    "requestId": "req_abc123"
  }
}
```


エラー時（404 Not Found）


```
{
  "success": false,
```

```
  "error": {
    "code": "RESOURCE_NOT_FOUND",
    "message": "Project not found or does not belong to your organization"
  },
  "meta": {
    "timestamp": "2026-01-27T10:30:00Z",
    "requestId": "req_abc123"
  }
}
```

ビジネスロジック

1. **バリデーション（issueService.validateIssueData）**
   - `title` が1-200文字であることを確認
   - `description` が1-2000文字であることを確認
   - `issueType` が0以上の整数であることを確認
   - `issueType` が `code_master` テーブル（`code_division='issue_type'`）の `code_value` に存在することを確認
   - `targetType` が `"project"`, `"task"`, `"transition"` のいずれかであることを確認

2. **targetType の数値変換（issueService.convertTargetTypeToNumber）**
   - `"project"` → `1`
   - `"task"` → `2`
   - `"transition"` → `3`

3. **問題登録**
   - `issue` テーブルに新規レコードを挿入（`issues/create.sql`）
   - `issue_reporter_id` に現在のユーザーIDを設定
   - `status` を `"unresolved"` で初期化
   - `organization_id` を設定（マルチテナント分離）

データベーステーブル

使用テーブル

1. **issue**（主テーブル）
   - organization_id, project_id, issue_id, method_id, issue_target（数値: 1=project, 2=task, 3=transition）
   - task_no, transition_id, issue_title, issue_type, issue_contents
   - status, issue_reporter_id, issue_report_at, issue_completer_id, issue_complate_at
   - create_user_id, create_at, update_user_id, update_at

2. **code_master**（issueType検証）
   - code_division='issue_type', code_value

SQL例（issues/create.sql）

```
INSERT INTO issue (
  organization_id, project_id, issue_id, method_id, issue_target, task_no, transition_id,
  issue_title, issue_type, issue_contents, status,
  issue_reporter_id, issue_report_at, issue_completer_id, issue_complate_at,
  create_user_id, create_at, update_user_id, update_at
) VALUES (
  $1, $2, $3, $4, $5, $6, $7, $8, $9, $10, 'unresolved',
  $11, NOW(), null, null,
  $11, NOW(), $11, NOW()
) RETURNING *;
```

セキュリティ

- [ ] organization_id フィルタ必須（マルチテナント分離）
- [ ] JWT認証必須
- [ ] Worker以上の権限が必要（`checkPermission('050000', 1)`）
- [ ] SQLインジェクション対策（Parameterized Queries）

- [ ] 文字列長制限の強制

---

2. コメント投稿API

概要

既存の問題にコメントを追加します。

エンドポイント

```
POST /api/v1/issues/comments
```

認証要件

- JWT認証必須
- ロール: Worker以上（`checkPermission('050000', 1)`）
- テナント分離必須

ミドルウェアチェーン

```
router.post('/comments',
  authMiddleware,
  tenantIsolation,
  checkPermission('050000', 1),
  [...validationRules],
  handleValidationErrors,
  issueController.addComment
);
```

リクエスト

リクエストボディ

```
{
  "projectName": "sample_project",
  "issueId": "issue_001",
  "comment": "在庫管理システムを確認したところ、試薬Aは明日入荷予定です。"
}
```

パラメータ	型	必須	説明	制約
projectName	string	○	プロジェクト名	空文字列ではない
issueId	string	○	問題ID	空文字列ではない
comment	string	○	コメント内容	1-2000文字

レスポンス

成功時（201 Created）

```
{
  "success": true,
  "data": {
    "projectId": "550e8400-e29b-41d4-a716-446655440000",
    "issueId": "issue_001",
    "sequenceNo": 1,
    "comment": "在庫管理システムを確認したところ、試薬Aは明日入荷予定です。",
    "authorId": "880e8400-e29b-41d4-a716-446655440003",
    "authorName": "山田太郎",
    "commentedAt": "2026-01-27T11:00:00Z"
  },
  "meta": {
    "timestamp": "2026-01-27T11:00:00Z",
    "requestId": "req_def456"
  }
}
```

エラー時（404 Not Found）

```
{
  "success": false,
  "error": {
```

```
    "code": "RESOURCE_NOT_FOUND",
    "message": "Issue not found or does not belong to your organization"
  },
  "meta": {
    "timestamp": "2026-01-27T11:00:00Z",
    "requestId": "req_def456"
  }
}
```

ビジネスロジック

1. **バリデーション**
   - `comment` が1-2000文字であることを確認
2. **プロジェクト確認**
   - `projectName` からプロジェクトIDを取得（`issues/getProjectIdByName.sql`）
   - 存在しない場合は 404 を返却
3. **問題存在確認**
   - `issueId` が自組織に属する問題であることを確認（`findById`）
4. **コメント登録（トランザクション）**
   - `issue_comment/addCommentCheckIssue.sql` で問題をFOR UPDATEロック
   - `issue_comment/addCommentGetNextSeq.sql` でシーケンス番号を採番
   - `issue_comment/addCommentInsert.sql` でコメントを挿入
   - `issue_comment/addCommentGetAuthorName.sql` で投稿者名を取得

データベーステーブル

使用テーブル

1. **issue_comment**（主テーブル）

- organization_id, project_id, issue_id, sequence_no, comment
- author_id, commented_at, create_user_id, create_at, update_user_id, update_at

2. **issue**（存在確認・FOR UPDATEロック用）
3. **projects**（プロジェクト名→ID変換）
4. **organization_user**（投稿者名取得）

SQL例

```
-- トランザクション内で実行

-- 1. 問題存在確認（FOR UPDATE）
SELECT issue_id, organization_id, project_id FROM issue
WHERE issue_id = $1 AND organization_id = $2 FOR UPDATE;

-- 2. シーケンス番号採番
SELECT COALESCE(MAX(sequence_no), 0) + 1 AS next_seq
FROM issue_comment
WHERE organization_id = $1 AND issue_id = $2;

-- 3. コメント挿入
INSERT INTO issue_comment (
  organization_id, project_id, issue_id, sequence_no, comment,
  author_id, commented_at, create_user_id, create_at, update_user_id, update_at
) VALUES ($1, $2, $3, $4, $5, $6, NOW(), $6, NOW(), $6, NOW())
RETURNING *;

-- 4. 投稿者名取得
SELECT ou.user_name FROM organization_user ou
WHERE ou.organization_id = $1 AND ou.user_id = $2;
```

セキュリティ

- [ ] organization_id フィルタ必須
- [ ] 自組織の問題のみコメント可能
- [ ] SQLインジェクション対策
- [ ] トランザクション制御（排他ロック）

---

3. 問題一覧取得API

概要

問題一覧を取得します。プロジェクト単位でのフィルタリングに加え、メソッド・タスク・トランジション単位でのオプションフィルタリングに対応し、問題に紐づくコメントも親子関係で取得します。

エンドポイント

```
GET /api/v1/issues
```

認証要件

- JWT認証必須
- ロール: Worker以上（ `checkPermission('050000', 0)` ）
- テナント分離必須

ミドルウェアチェーン

```
router.get('/',
  authMiddleware,
  tenantIsolation,
  checkPermission('050000', 0),
  [...validationRules],
  handleValidationErrors,
  issueController.list
);
```

リクエスト

クエリパラメータ

パラメータ	型	必須	デフォルト	説明
projectId	string	○	-	プロジェクトIDでフィルタ（必須）
methodId	string	△	-	メソッドIDでフィルタ（オプション。省略時はプロジェクト全件）
taskNo	integer	△	-	タスク連番でフィルタ（オプション。issue_target=2の問題を絞り込む）
transitionId	string	△	-	トランジションIDでフィルタ（オプション。issue_target=3の問題を絞り込む）
status	string	△	-	ステータスでフィルタ（オプション: `"unresolved"` / `"resolved"`）
issueType	integer	△	-	問題タイプでフィルタ（オプション）

注意: `methodId`、`taskNo`、`transitionId` はすべてオプションです。省略した場合はプロジェクト全体の問題を返します。

リクエスト例

```
GET /api/v1/issues?projectId=550e8400-e29b-41d4-a716-446655440000
GET /api/v1/issues?projectId=550e8400-e29b-41d4-a716-
446655440000&methodId=method_001&taskNo=1&transitionId=transition_001
GET /api/v1/issues?projectId=550e8400-e29b-41d4-a716-446655440000&status=unresolved
```

レスポンス

成功時（200 OK）


```
{
  "success": true,
  "data": [
    {
      "id": "issue_001",
      "issueType": 1,
      "issueTarget": 3,
      "projectId": "550e8400-e29b-41d4-a716-446655440000",
      "projectName": "Sample Project",
      "methodId": "method_001",
      "taskNo": 1,
```

```
      "transitionId": "transition_001",
      "title": "試薬Aの在庫不足",
      "contents": "実験に必要な試薬Aの在庫が不足しています。",
      "status": "unresolved",
      "reportedBy": "880e8400-e29b-41d4-a716-446655440003",
      "reportedByName": "山田太郎",
      "reportedAt": "2026-01-27T10:30:00Z",
      "completedBy": null,
      "completedByName": null,
      "completedAt": null,
      "comments": [
        {
          "sequenceNo": 1,
          "comment": "在庫管理システムを確認したところ、試薬Aは明日入荷予定です。",
          "authorId": "880e8400-e29b-41d4-a716-446655440005",
          "authorName": "佐藤花子",
          "commentedAt": "2026-01-27T11:00:00Z"
        },
        {
          "sequenceNo": 2,
          "comment": "承知しました。入荷次第、実験を再開します。",
          "authorId": "880e8400-e29b-41d4-a716-446655440003",
          "authorName": "山田太郎",
          "commentedAt": "2026-01-27T11:30:00Z"
        }
      ]
    }
  ],
  "meta": {
    "timestamp": "2026-01-27T12:00:00Z",
    "requestId": "req_ghi789"
  }
}
```


ビジネスロジック（issueRepository.listIssuesWithComments）

1. **2段階クエリ（N+1問題回避）**

   Query 1: 問題一覧取得（`issues/listIssuesWithCommentsIssues.sql`）

   - `organization_id`（$1）と`project_id`（$2）は必須条件
   - 以下のオプションフィルタを動的に追加:
     - `methodId`指定時: `AND i.method_id = $N`

    - `taskNo` 指定時: `AND (i.issue_target != 2 OR i.task_no = $N)`（issue_target=2の問題のみ絞り込み）
    - `transitionId` 指定時: `AND (i.issue_target != 3 OR i.transition_id = $N)`（issue_target=3の問題のみ絞り込み）
    - `status` 指定時: `AND i.status = $N`
    - `issueType` 指定時: `AND i.issue_type = $N`
  - 固定ソート: `created_at DESC`

Query 2: コメント一括取得（`issues/listIssuesWithCommentsComments.sql`）

  - Query 1で取得した問題IDの配列を使用してコメントを一括取得
  - `sequence_no ASC` でソート
  - コメントを `issueId` でグループ化して各問題の `comments` プロパティに設定

2. **ページネーション**: なし（全件取得）

バリデーション

- [ ] projectId は必須（空文字列ではない）
- [ ] methodId はオプション（指定時は文字列）
- [ ] taskNo はオプション（指定時は1以上の整数）
- [ ] transitionId はオプション（指定時は文字列）
- [ ] status はオプション（指定時は `"unresolved"` または `"resolved"`）
- [ ] issueType はオプション（指定時は整数）

データベーステーブル

使用テーブル

1. **issue**（主テーブル）
2. **issue_comment**（コメント取得）
3. **projects**（JOIN: プロジェクト名取得）

4. **organization_user**（JOIN: 報告者名・完了者名・コメント投稿者名取得）

SQL例（issues/listIssuesWithCommentsIssues.sql ベース）

```
-- 問題一覧取得（動的フィルタ）
SELECT
  i.issue_id AS id,
  i.issue_type AS "issueType",
  i.issue_target AS "issueTarget",
  i.project_id AS "projectId",
  p.project_name AS "projectName",
  i.method_id AS "methodId",
  i.task_no AS "taskNo",
  i.transition_id AS "transitionId",
  i.issue_title AS title,
  i.issue_contents AS contents,
  i.status,
  i.issue_reporter_id AS "reportedBy",
  reporter.user_name AS "reportedByName",
  i.issue_report_at AS "reportedAt",
  i.issue_completer_id AS "completedBy",
  completer.user_name AS "completedByName",
  i.issue_complate_at AS "completedAt"
FROM issue i
JOIN projects p ON i.project_id = p.project_id AND p.organization_id = i.organization_id
LEFT JOIN organization_user reporter ON i.issue_reporter_id = reporter.user_id AND
reporter.organization_id = i.organization_id
LEFT JOIN organization_user completer ON i.issue_completer_id = completer.user_id AND
completer.organization_id = i.organization_id
WHERE i.organization_id = $1
  AND i.project_id = $2
-- オプションフィルタが動的に追加される
ORDER BY i.create_at DESC;
```

セキュリティ

- [ ] organization_id フィルタ必須
- [ ] SQLインジェクション対策（Parameterized Queries）

---

4. 問題詳細取得API

概要

問題の詳細情報を取得します。

エンドポイント

```
GET /api/v1/issues/:issueId
```

認証要件

- JWT認証必須
- テナント分離必須
- 権限チェックなし（`authMiddleware` + `tenantIsolation` のみ）

ミドルウェアチェーン

```
router.get('/:issueId',
  authMiddleware,
  tenantIsolation,
  [
    param('issueId').isUUID().withMessage('issueIdはUUID形式である必要があります'),
  ],
  issueController.getById
);
```

リクエスト

パスパラメータ

パラメータ	型	必須	説明
issueId	UUID	○	問題ID（UUID形式）

リクエスト例

```
GET /api/v1/issues/770e8400-e29b-41d4-a716-446655440002
```

レスポンス

成功時（200 OK）


```
{
  "success": true,
  "data": {
    "organization_id": "org-uuid",
    "project_id": "550e8400-e29b-41d4-a716-446655440000",
    "issue_id": "770e8400-e29b-41d4-a716-446655440002",
    "method_id": "method_001",
    "issue_target": 3,
    "task_no": 1,
    "transition_id": "transition_001",
    "issue_title": "試薬Aの在庫不足",
    "issue_type": 1,
    "issue_contents": "実験に必要な試薬Aの在庫が不足しています。追加発注が必要です。",
    "status": "unresolved",
    "issue_reporter_id": "880e8400-e29b-41d4-a716-446655440003",
    "issue_report_at": "2026-01-27T10:30:00Z",
    "issue_completer_id": null,
    "issue_complate_at": null,
    "create_at": "2026-01-27T10:30:00Z",
    "update_at": "2026-01-27T10:30:00Z"
  },
  "meta": {
    "timestamp": "2026-01-27T12:00:00Z",
    "requestId": "req_jkl012"
  }
}
```


エラー時（400 Bad Request - UUID形式不正）


```
{
  "success": false,
  "error": {
    "code": "VALIDATION_INVALID_FORMAT",
    "message": "issueIdはUUID形式である必要があります",
    "details": [
```

```
      {
        "field": "issueId",
        "message": "UUID形式が必要です",
        "code": "INVALID_FORMAT"
      }
    ]
  },
  "meta": {
    "timestamp": "2026-01-27T12:00:00Z",
    "requestId": "req_jkl012"
  }
}
```

エラー時（404 Not Found）

```
{
  "success": false,
  "error": {
    "code": "RESOURCE_NOT_FOUND",
    "message": "問題報告が見つかりません"
  },
  "meta": {
    "timestamp": "2026-01-27T12:00:00Z",
    "requestId": "req_jkl012"
  }
}
```

ビジネスロジック

1. **UUID形式バリデーション**
   - `issueId` がUUID形式であることをコントローラー側でも確認（400を返却）
2. **問題詳細取得**
   - `issues/findById.sql` で `issue_id` と `organization_id` を指定して取得
   - 存在しない場合は 404 を返却

データベーステーブル

使用テーブル

1. **issue**（主テーブル）

SQL例（issues/findById.sql）

```
SELECT * FROM issue
WHERE issue_id = $1 AND organization_id = $2;
```

セキュリティ

- [ ] organization_id フィルタ必須
- [ ] SQLインジェクション対策

---

5. 問題ステータス変更API

概要

問題のステータスを「未解決」(0) または「解決済」(1) に変更します。

エンドポイント

```
PATCH /api/v1/issues/status
```

認証要件

- JWT認証必須
- ロール: Worker以上（`checkPermission('050000', 1)`）

- テナント分離必須

ミドルウェアチェーン

```
router.patch('/status',
  authMiddleware,
  tenantIsolation,
  checkPermission('050000', 1),
  [...validationRules],
  handleValidationErrors,
  issueController.updateStatus
);
```

リクエスト

リクエストボディ

```
{
  "projectName": "sample_project",
  "issueId": "issue_001",
  "status": 1
}
```

パラメータ	型	必須	説明	制約
projectName	string	○	プロジェクト名	空文字列ではない
issueId	string	○	問題ID	1-50文字
status	integer	○	ステータス	`0`（未解決）または `1`（解決済）

重要: `status` は文字列ではなく **integer** です。`0` が未解決、`1` が解決済を表します。

レスポンス

成功時（200 OK） - 解決済に変更した場合

```
{
  "success": true,
  "data": {
    "organization_id": "org-uuid",
    "project_id": "project-uuid",
    "issue_id": "issue_001",
    "status": "resolved",
    "issue_completer_id": "880e8400-e29b-41d4-a716-446655440003",
    "issue_complate_at": "2026-01-27T15:00:00Z",
    "update_at": "2026-01-27T15:00:00Z"
  },
  "meta": {
    "timestamp": "2026-01-27T15:00:00Z",
    "requestId": "req_mno345"
  }
}
```


成功時（200 OK） - 未解決に戻した場合


```
{
  "success": true,
  "data": {
    "organization_id": "org-uuid",
    "project_id": "project-uuid",
    "issue_id": "issue_001",
    "status": "unresolved",
    "issue_completer_id": null,
    "issue_complate_at": null,
    "update_at": "2026-01-27T15:00:00Z"
  },
  "meta": {
    "timestamp": "2026-01-27T15:00:00Z",
    "requestId": "req_mno345"
  }
}
```


エラー時（400 Bad Request）


```
{
  "success": false,
  "error": {
```

```
    "code": "VALIDATION_ERROR",
    "message": "無効なステータスです。許可された値: 0（未解決）, 1（解決済）"
  },
  "meta": {
    "timestamp": "2026-01-27T15:00:00Z",
    "requestId": "req_mno345"
  }
}
```

エラー時（404 Not Found）

```
{
  "success": false,
  "error": {
    "code": "RESOURCE_NOT_FOUND",
    "message": "問題報告が見つかりません"
  },
  "meta": {
    "timestamp": "2026-01-27T15:00:00Z",
    "requestId": "req_mno345"
  }
}
```

ビジネスロジック

1. **プロジェクト確認**
   - `projectName` からプロジェクトIDを取得（`issues/getProjectIdByName.sql`）
   - 存在しない場合は 404 を返却
2. **問題存在確認**
   - `issueId` が自組織に属する問題であることを確認（`findById`）
   - 存在しない場合は 404 を返却
3. **ステータスバリデーション**
   - `status` が `0` または `1` であることを確認（それ以外は 400）

4. **ステータス更新**（`issues/updateStatus.sql`）
    - `status` の数値をSQLファイルに渡して更新
    - 更新後のレコードを RETURNING で返却

データベーステーブル

使用テーブル

1. **issue**（更新対象）
    - status, issue_completer_id, issue_complate_at, update_at

SQL例（issues/updateStatus.sql）

```
UPDATE issue
SET
  status = CASE WHEN $1 = 1 THEN 'resolved' ELSE 'unresolved' END,
  issue_completer_id = CASE WHEN $1 = 1 THEN $2 ELSE NULL END,
  issue_complate_at = CASE WHEN $1 = 1 THEN NOW() ELSE NULL END,
  update_at = NOW()
WHERE issue_id = $3 AND organization_id = $4
RETURNING *;
```

セキュリティ

- [ ] organization_id フィルタ必須
- [ ] 自組織の問題のみ更新可能
- [ ] JWT認証必須
- [ ] Worker以上の権限が必要（`checkPermission('050000', 1)`）
- [ ] SQLインジェクション対策（Parameterized Queries）

🔗 関連ドキュメント

API設計書

- api_list.md - API一覧対応表
- api_project_management.md - プロジェクト管理API
- api_progress_monitoring.md - 進捗確認API

規約・ガイドライン

- API設計ポリシー - API設計の統一ルール
- バックエンド規約 - 実装規約
- セキュリティ規約 - セキュリティ要件

📊 実装ファイルパス

ルートファイル

```
backend/src/routes/issueRoutes.ts
```

コントローラーファイル

```
backend/src/controllers/issueController.ts
```

サービスファイル

```
backend/src/services/issueService.ts
```

リポジトリファイル

```
backend/src/repositories/issueRepository.ts
```

SQLファイル

```
backend/src/sql/queries/issues/create.sql
backend/src/sql/queries/issues/findById.sql
backend/src/sql/queries/issues/updateStatus.sql
backend/src/sql/queries/issues/listIssuesWithCommentsIssues.sql
backend/src/sql/queries/issues/listIssuesWithCommentsComments.sql
backend/src/sql/queries/issues/getProjectIdByName.sql
backend/src/sql/queries/issue_comment/addCommentCheckIssue.sql
backend/src/sql/queries/issue_comment/addCommentGetNextSeq.sql
backend/src/sql/queries/issue_comment/addCommentInsert.sql
backend/src/sql/queries/issue_comment/addCommentGetAuthorName.sql
```

テストファイル

```
backend/tests/routes/issueRoutes.test.ts
backend/tests/services/issueService.test.ts
backend/tests/controllers/issueController.test.ts
```

✅ セキュリティチェックリスト

認証・認可

- [ ] すべてのエンドポイントでJWT認証が必須（`authMiddleware`）
- [ ] `tenantIsolation` ミドルウェアが適用されている
- [ ] `checkPermission('050000', 0 or 1)` で適切な権限が設定されている

マルチテナント分離

- [ ] すべてのDBクエリにorganization_idフィルタが含まれる
- [ ] クロステナントアクセスが不可能

入力検証

- [ ] すべてのパラメータがバリデーションされている（`express-validator`）
- [ ] `handleValidationErrors` でバリデーションエラーを処理
- [ ] Parameterized Queriesを使用（SQLインジェクション対策）
- [ ] 文字列長制限が強制されている

エラーハンドリング

- [ ] 統一されたエラーレスポンス形式
- [ ] エラーメッセージに機密情報が含まれない
- [ ] 適切なHTTPステータスコードを返す

ログ記録

- [ ] エラー発生時の詳細ログ（`logError`）
- [ ] ステータス変更操作の監査ログ（`logger.info`）

📝 変更履歴

バージョン	日付	変更内容	担当者
1.1.0	2026-03-12	ソースコード実装に基づき仕様書を修正	Cline
		- API #2 コメント投稿: エンドポイントを `/issues/:issueId/comments` → `/issues/comments` に修正	
		- API #2 コメント投稿: ミドルウェアを `checkPermission('050000', 1)` に修正	

		- API #3 問題一覧取得: `methodId` / `taskNo` / `transitionId` をオプションに変更	
		- API #3 問題一覧取得: フィルタリングロジックを実装（2段階クエリ、N+1問題回避）に合わせて修正	
		- API #3 問題一覧取得: ミドルウェアを `checkPermission('050000', 0)` に修正	
		- API #4 問題詳細取得: ミドルウェアを `authMiddleware` + `tenantIsolation` のみに修正（`checkPermission` なし）	
		- API #5 問題ステータス変更: `status` を string → integer（0=未解決, 1=解決済）に修正	
		- API #5 問題ステータス変更: ミドルウェアを `checkPermission('050000', 1)` に修正（Worker以上）	
		- 全API: バリデーションに `handleValidationErrors` ミドルウェアを追加	
		- 全API: `title` 最大文字数を1-200文字、`description` を1-2000文字に修正	
		- SQL例・実装ファイルパスをソースコードに合わせて更新	
1.0.0	2026-01-27	初版作成	Cline
		- 問題管理API 5エンドポイントの設計書作成	
		- 実装済み（テスト103件すべてPASS）	

注意: この設計書は実装済みAPIの仕様を文書化したものです。実装の詳細については、実際のソースコード（`backend/src/routes/issueRoutes.ts` 等）を参照してください。